Automatic Root Cause Analysis via Large Language Models for Cloud Incidents

CS 395T: Principles of Learned Systems

Presenter: Sanika Nandpure

Root Cause Analysis in Cloud Systems

- Incident life-cycle:
  - Detection
  - Triaging
  - Diagnosis
  - Mitigation
- Issues with traditional/manual RCA:
  - Manual collection, parsing, analysis of high-volume data
  - Data is complex and often unstructured
  - Noisy, incomplete/inconsistent data
  - Information spectrum (not enough or too much information)
- Troubleshooting Guides (TSGs)
  - Need to be manually updated
  - Contain outdated or insufficient information (cannot cover all possibilities)
- Key challenge:
  - **Efficiently collecting and interpreting comprehensive, incident-specific data within a limited time frame**

Why LLMs for RCA?

- LLMs can be used to parse through and interpret large amounts of data
  - Can determine which information is directly relevant to the task
  - Can organize/summarize information in a more interpretable manner
- Challenges:
  - Lack domain-specific knowledge
  - Data is highly complex and comes in various formats (logs, telemetry, traces, etc.)
  - Fine-tuning is expensive and requires huge volume of training samples
  - Continuously updating models with new knowledge is challenging
  - Unreliable outputs (hallucination)

Types of incident data

- Traces (call-graphs)
  - Tree-structured data, follows the path of a single request through the system
- Logs
  - Semi-structured, textual; record various hardware and software events
- Metrics
  - Service status, user-perceived metrics (e.g., latency, CPU utilization, etc.); forms time-series data.
- Different data sources offer different perspectives on the issue ⇒ need to consider them together
- Can be inconsistent, noisy, high-volume

Table 1. Examples of cloud incidents in different root cause categories.

No.	Sev.	Scope	Category	Occur.	Symptom	Cause
1	1	Forest	AuthCertIssue	3	Tokens for requesting services were not able to be created. Several services reported users experiencing outages.	A previous invalid certificate overrided the existing one due to misconfiguration.
2	2	Machine	HubPortExhaustion	27	A single server failed to do DNS resolution for the incoming packages.	The UDP hub ports on the machine had been run out.
3	2	Forest	DeliveryHang	6	Mailbox delivery service hang for a long time.	Number of messages queued for mailbox delivery exceeded the limit.
4	2	Forest	CodeRegression	15	An SMTP authentication component's availability dropped.	Bug in the code.
5	2	Forest	CertForBogusTenants	11	The number of concurrent server connections exceeded a limit.	Spammers abused the system by creating a lot of bogus tenants with connectors using a certificate domain.
6	1	Forest	MaliciousAttack	2	Forest-wide processes crashed over threshold.	Active exploit was launched in remote PowerShell by serializing malicious binary blob.
7	2	Forest	UseRouteResolution	9	Poisoned messages sent to the forest made the system unhealthy.	A configuration service was unable to update the settings leading to the crash.
8	2	Forest	FullDisk	2	Many processes crashed and threw IO exceptions.	A specific disk was full.
9	2	Forest	InvalidJournaling	11	Messages stuck in submission queue for a long time.	The customer set an invalid value for the Transport config and caused TenantSettingsNotFoundException.
10	3	Forest	DispatcherTaskCancelled	22	Normal priority messages across a forest had been queued in submission queues for a long time.	Network problem caused the authentication service to be unreachable.

Insights

- Determining the root cause based on a single data source can be challenging
- Incidents stemming from similar/identical root causes reappear within a short period due to unresolved root causes, external dependencies, or vulnerability exploitation
- Incidents with new root causes occur frequently and are challenging to analyze since TSGs don’t exist

RCACopilot (high-level view)

Two stages:

- Diagnostic information collection stage
  - Incident is parsed + matched to pre-defined incident handler, relevant data gathered
- Root cause prediction stage
  - Apply LLM to predict likely root cause category

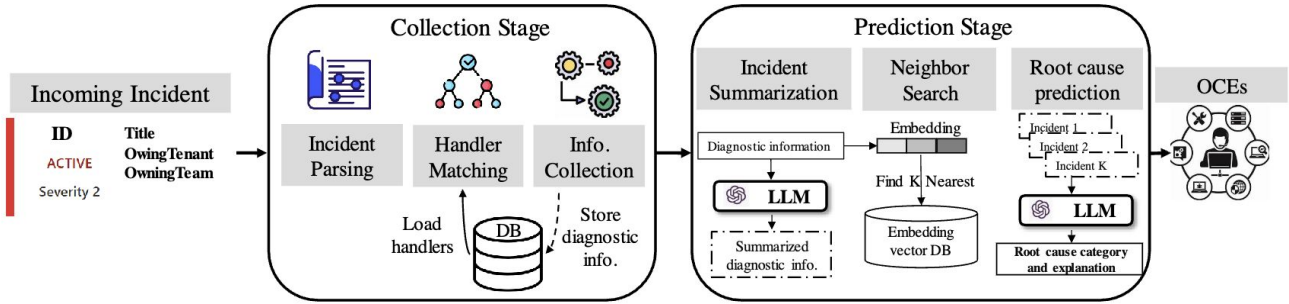


Figure 4. RCACopilot architecture.

Stage 1: Diagnostic information collection

- Incident is parsed, matched to pre-defined incident handler
  - Each handler is tailored for a specific alert type
- Handler pre-defines what information is “relevant”
- Gather/parse relevant data
  - RCA Copilot can do common checks such as evaluating provisioning status, analyzing thread stacks

Incident handler

- Incident handler = workflow, contains sequence of actions
- OCEs build and modify handlers using domain expertise
- Similar to decision tree's control flow
- Actions are designed to be reusable across all handlers
- Handler includes:
  - (1) scope switching action
  - (2) query action
  - (3) mitigation action

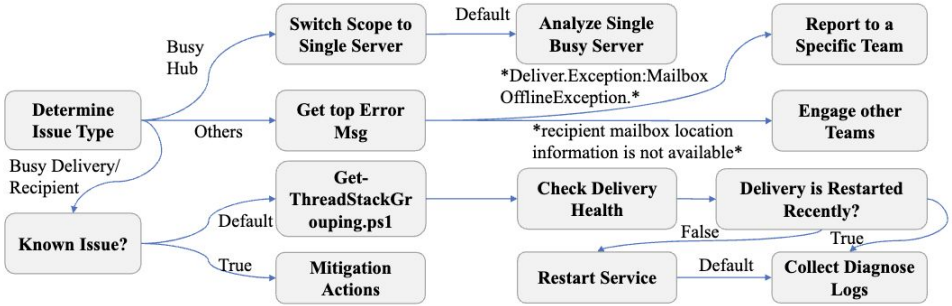


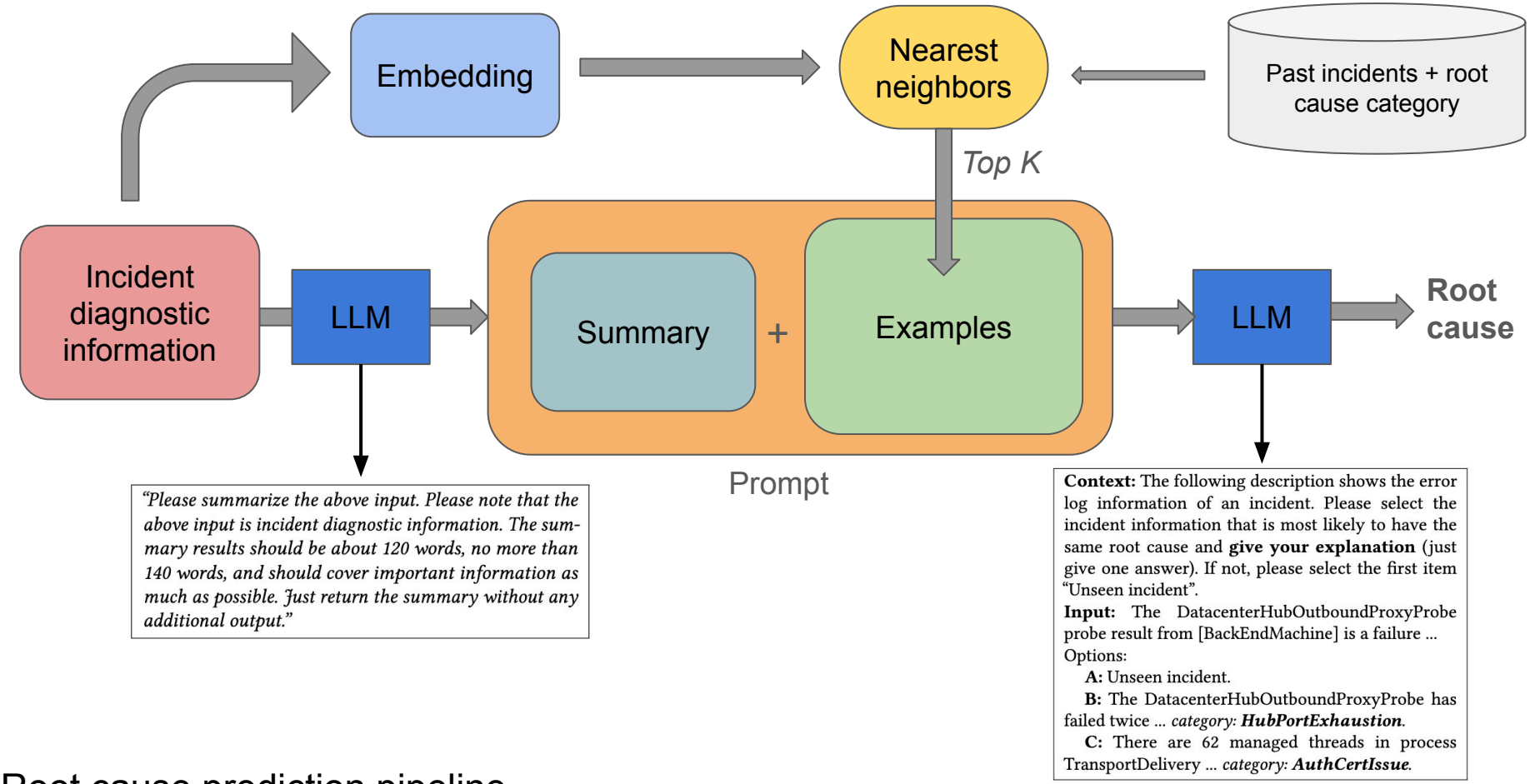
Figure 5. A RCACopilot handler for too many messages stuck in the delivery queue alert.

Incident handler

- **Scope switching action** - allows adjustments to data collection scope based on specific needs of each incident ⇒ enables targeted data collection at right granularity
  - Forest level, machine level, etc.
- **Query action -** query data from different sources, outputs data as a key-value pair table
- **Mitigation action -** solution steps to resolve the incident

Stage 2: Predicting Root Cause

- CoT prompting: summary of current incident + examples of root cause categories of past incidents
- Use data on historical incidents, manually labelled by developers
- To find relevant examples, use embedding: map diagnostic information into embedding space
  - Distance between two vectors = semantic similarity of incidents
  - Find K nearest neighbor-incidents, use as examples in the prompt to LLM



Root cause prediction pipeline

Evaluation

- Micro-F1: 0.766, Macro-F1: 0.533
  - Outperforms baselines
- Can predict new root cause categories it has not seen before
- Can retrieve relevant diagnostic information in 15 seconds-14 minutes
- Diagnostic information collection module deployed across 30 teams @ Microsoft

Table 2. Effectiveness of different methods.

Method	F1-score		Avg. Time (s)	
	Micro	Macro	Train.	Infer.
FastText [61]	0.076	0.004	10.592	0.524
XGBoost [3]	0.022	0.009	11.581	1.211
Fine-tune GPT [1]	0.103	0.144	3192	4.262
GPT-4 Prompt	0.026	0.004	–	3.251
GPT-4 Embed.	0.257	0.122	1925	3.522
RCACopilot (GPT-3.5)	0.761	0.505	10.562	4.221
RCACopilot (GPT-4)	**0.766**	**0.533**	10.562	4.205

Table 3. Effectiveness of different prompt context for RCACopilot. ✓$^{sum.}$ stands for the summarized diagnostic information.

Data Source			F1-score	
AlertInfo	DiagnosticInfo	ActionOutput	Micro	Macro
	✓		0.689	0.510
	✓$^{sum.}$		**0.766**	**0.533**
✓			0.379	0.245
✓	✓		0.525	0.511
✓		✓	0.431	0.247
	✓	✓	0.501	0.449
✓	✓	✓	0.440	0.349

Thoughts/Discussion

- Not accurate enough for handling RCA independently
- Lots of human intervention still required; role of LLM is quite small
  - RCA Copilot is only as good as the incident handlers and past labelled incidents
  - Provides a good reusable framework for RCA for developers, but LLM does not play as significant of a role as advertised
- Use very simple prompting technique (CoT)
  - How would this perform w/ RAG, thought-template prompting, etc.?
- Would be interesting to see how this can be improved now with newer models, agentic systems, etc.